
The "Sidewalk Sabotage" Predictor

Jianping Chen Mario Gegprifti Daniel Soriano Anis Tarafdar Nicolas Hughes-Barrow

1. Abstract

New York City's 2015 Street Tree Census recorded over 680,000 living trees, along with whether each tree was associated with sidewalk damage. We built a classifier to predict that damage using tree characteristics such as species, diameter, health, and location. We evaluated logistic regression as a baseline, then moved to random forest, which we tuned via grid search over tree depth and feature sampling parameters. We also tested XGBoost and applied clustering to explore patterns in the data. Because missing a damaged sidewalk carries a higher cost than sending an inspector to check a false positive, we optimized for F2 score rather than F1. This metric weights recall higher than precision. Our final hyperparameter-tuned random forest model achieves an F2 of 0.7247 and a recall of 90.77% on held-out test data. We also confirmed that the model's predictions rely mainly on tree characteristics rather than variables that may have been recorded alongside the damage label, reducing concerns about bias in the data.

2. Introduction

2.1. Problem Definition

New York City manages over 680,000 street trees that provide essential environmental benefits. As these trees grow, their root systems interact with urban infrastructure causing the concrete sidewalk slabs to crack or lift. An important challenge for the Department of Parks and Recreation is predicting which trees are most likely to cause this damage so they can prioritize maintenance and safety inspections. The goal of this project is to develop a Machine Learning binary classification system that predicts whether a tree is currently associated with sidewalk damage based on its biological features, such as species and diameter, and geographical location.

The repository for this project can be found here: <https://github.com/alanchen8647/MTH4330>.

3. Background

Several projects have used the 2015 Street Tree Census dataset to predict infrastructure damage and tree health. However, these models' attempt to predict infrastructure

damage from street trees have often been limited by relying on overall accuracy metrics, which obscure their failure to identify the rare but crucial instances of actual damage (Stevens, 2016; Belyaev, 2023). To build more practical tools, research suggests shifting the focus from broad geographical hotspots to individual tree predictions, especially since neighborhood inequality directly correlates with poorer tree health and a higher probability of sidewalk damage (Stevens, 2016; Lin & Wang, 2021). Furthermore, improving these predictions requires incorporating often-overlooked features like a tree's specific species, which has proven to be just as significant a predictor of damage as traditional metrics like trunk thickness or curb proximity (Francis et al., 1996). Unlike prior work that attempts to predict future damage, our model focuses on classifying current damage, which is better suited to a static snapshot dataset like the 2015 Street Tree Census. While prior approaches have often focused on identifying high-risk neighborhoods or geographic hotspots, our model makes a prediction for each individual tree, which allows for more targeted inspection prioritization.

4. Dataset

4.1. Dataset Evaluation

The "2015 Street Tree Census - Tree Data" dataset consists of over 680,000 records collected during the TreeCount! 2015 Street Tree Census, which was conducted by volunteers and staff organized by NYC Parks & Recreation and partner organizations (NYC OpenData, 2015). Each row in the dataset represents an individual street tree in New York City and includes a unique `tree_id` as the primary identifying key. The entries include biological information such as species names (both scientific and common), the diameter of the tree and stump, perceived health status, and specific tree problems or causes observed by volunteers. Beyond biological data, the dataset contains geographic information, including latitude and longitude in decimal degrees, the street address closest to the tree, and the tree's placement relative to the curb. Our initial approach is to utilize this physical and biological information to predict which trees will eventually damage the sidewalk. Features we are considering for this model include geographic coordinates and species type, as well as environmental factors such as signs

Category	Features	
Biological	tree_dbh, spc_common	
Geographic	borough, nta_name, longitude	curb_loc, latitude, longitude
Health/Condition	health, steward	
Infrastructure	root_stone, root_other, trnk_light, brch_light, brch_other	root_grate, trunk_wire, trnk_other, brch_shoe, brch_other

Table 1. Feature table grouping potential features by category.

of stewardship and curb location. A full list of features we plan to use can be found in Table 1.

4.2. Data Processing and Preparation

Initial observation of the dataset revealed that approximately 4.62% of records contained a missing value in the target sidewalk feature. Further analysis confirmed these gaps were associated with the tree’s status, which is categorized as “Dead” or “Stump”. As these observations cannot provide meaningful data regarding sidewalk damage, we restricted our scope to “Alive” trees, effectively eliminating these null entries. This has resulted in a final row count of 652,166, of which 464,976 rows represent non-damaged sidewalks and 187,190 rows represent damaged sidewalks. We address this class imbalance in future. In addition, tree species were considered an important feature in light of biological factors; we mainly looked at `spc_common` (common name of tree species). However, the raw data contains over 130 unique species, representing high categorical cardinality that could lead to model inefficiency. To streamline the feature space, we retained only the top n most frequent species, grouping the remainder into an “Other” category. Iterative testing on our baseline model indicated that increasing n beyond the top 20 did not yield significant performance gains. Under such a setting, approximately 18% of the tree records will be categorized as “Other”. We decided to stick with the top 20 species to maintain efficiency without sacrificing important biological signals that would otherwise reduce accuracy.

4.3. Data Splits

To optimize model performance, we utilized a three-way data split of 80% train, 10% development, and 10% test. The development set is specifically used for cutoff optimization and hyperparameter tuning in order to keep the model unbiased towards the test set. The test set is reserved for the final model.

4.4. Evaluation Metrics

In the context of our project goal, the cost of a false negative is higher than a false positive. A false negative represents an undetected hazard, which poses an ongoing public safety risk and potential liability for the city, whereas a false positive merely results in a redundant site inspection. We chose to prioritize Recall and Precision over Accuracy to ensure maximum detection of sidewalk hazards. Because we weigh the cost of a false negative much higher than a false positive, we use an $F\beta$ -score with $\beta = 2$ as our primary metric. F2-score weights recall twice as much as it weights precision. This allows us to prioritize reducing false negatives, while still keeping false positives within a manageable range.

5. Methods

5.1. Logistic Regression

The logistic regression model was implemented using R’s built-in `glm` function with `family = "binomial"`. With sidewalk as the binary target variable (1 = Damage, 0 = No Damage), the model was trained on 18 features including tree diameter, species, health status, root/trunk/branch problems, borough, curb location, latitude, and longitude.

Logistic regression models the probability of damage as:

$$P(\text{damage}) = \frac{1}{1 + e^{-(X\beta)}}$$

where X is the design matrix of features and β contains the learned coefficients. Each feature receives a coefficient indicating its contribution to the log-odds of sidewalk damage. For categorical variables like species and borough, each level is one-hot encoded and receives its own coefficient.

We chose Logistic Regression as our baseline because it is interpretable and has been used successfully on similar tree-infrastructure datasets (Francis et al., 1996). However, it assumes linear relationships between features and log-odds, which limits its ability to capture complex interactions between tree characteristics and damage.

5.2. Random Forest

For the random forest model, we used the `ranger` package with `probability = TRUE` to output class probabilities rather than hard predictions. A random forest of 500 trees was trained, as this is the default and yielded stable results higher tree counts showed minimal improvement at increased computation time.

Random forest works by training multiple decision trees on bootstrapped samples of the data. At each split (controlled by `min.node.size`), only a random subset of features (controlled by `mtry`) is considered. The final prediction is the

Model	Accuracy	Precision	Recall	F2-Score
LR	0.7588	0.6317	0.3734	0.4067
RF	0.7723	0.6681	0.4036	0.4383
XGBoost	0.7651	0.6459	0.3935	0.4269

Table 2. Performance metrics of the baseline models on the dev set. Random Forest outperformed both Logistic Regression and XGBoost.

average probability across all trees:

$$P(\text{damage}) = \frac{1}{500} \sum_{i=1}^{500} \hat{P}_i(\text{damage})$$

Unlike logistic regression, random forest handles the mix of continuous (tree diameter, latitude, longitude), categorical (species, borough, health), and binary (`root_stone`, `trunk_wire`) features natively without requiring explicit encoding. In order to use this feature, we stored all categorical features as factors before training.

5.3. XGBoost

The XGBoost model was implemented using R’s XGBoost package with objective “binary:logistic”. Since XGBoost requires a fully numeric input matrix, categorical features such as species, borough, and health were one-hot encoded using `sparse.model.matrix`. The neighborhood variable `nta_name` was excluded because it has over 200 unique neighborhood values, which made one-hot encoding impractical. Class imbalance was addressed using the `scale_posweight` parameter, set to the ratio of negative to positive examples in the training set (approximately 2.48).

XGBoost is a gradient boosting method that builds trees sequentially, where each new tree is trained to correct the errors made by the previous ones. The final prediction is the sum of the outputs of all trees passed through a sigmoid function:

$$P(\text{damage}) = \sigma \left(\sum_k k = 1^K f_k(x) \right)$$

where f_k represents the k -th tree and σ is the sigmoid function that maps the output to a probability. We tested XGBoost because it often outperforms random forest on structured tabular data by focusing more on difficult examples across iterations.

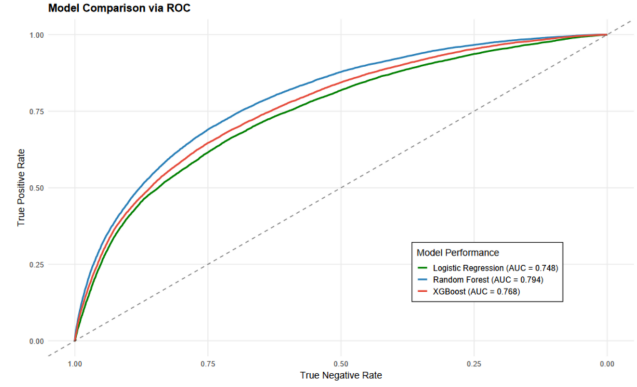


Figure 1. ROC and AUCs of the baseline Logistic Regression, Random Forest, and XGBoost model. The ROC and AUC for the Random Forest model outperforms both of the other models.

6. Experiments

6.1. Baseline Model Comparison

Initial experiments compared baseline implementations of logistic regression, random forest, and XGBoost models. The random forest baseline outperformed logistic regression and XGBoost across all primary metrics, achieving an F2-score of 0.4383 compared to the logistic regression’s 0.4067 and XGBoost’s 0.4269, as seen in Table 2. This is reflected in the ROC and AUC of each model, as shown in Figure 1. This performance gap suggests that the relationship between tree characteristics and sidewalk damage is more non-linear. While logistic regression assumes a linear log-odds relationship, the random forest ensemble more effectively captures complex interactions between a tree’s biological traits and its geographical location. Random forest may also outperform XGBoost simply due to the inherent randomness and noise of the dataset.

6.2. Cutoff Optimization

Given our highly imbalanced dataset, the default 0.50 classification cutoff required optimization. We performed a grid search on the development set, testing cutoffs from 0.10 to 0.90 in 0.01 increments. We evaluated the metrics of each cutoff selected the cutoff that yielded the highest F2-score. Cutoff optimization for the highest F2-score was performed on every model that we trained, including for each of the individual models trained during hyperparameter tuning.

6.3. Class Weights

To further address the high class imbalance of the dataset, we applied cost-sensitive learning using a relative class weight ratio. We designated the majority “No Damage” class as the baseline with a weight of 1, while assigning

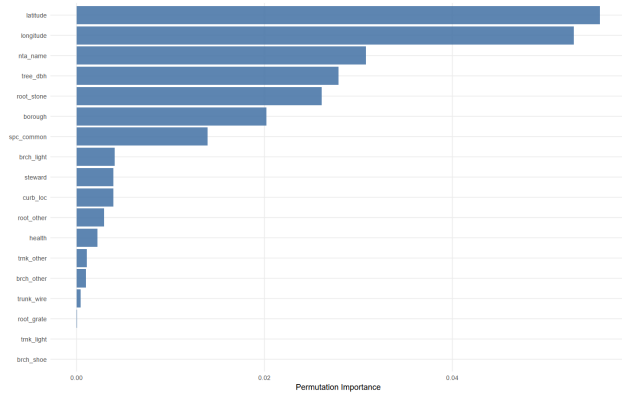


Figure 2. Feature importance according to permutations on the random forest model.

the "Damage" class a weight of approximately 2.48 based on the inverse ratio of their frequencies in the training set. These weights were applied to the linear regression models, random forest models via the `case.weights` parameter, and to the XGBoost models via the `scale_pos_weight` parameter to ensure that the model penalized the misclassification of sidewalk hazards more heavily. By scaling the importance of the minority class, we enabled the models to prioritize identifying the biological and geographical signals associated with infrastructure damage.

6.4. Feature Engineering and Reduction

Permutation importance results reveal that spatial coordinates are the most influential predictors, as shown in Figure 2. We intended to focus on lessening the importance of these indicators in favor of engineered features that focus more on biological features.

We conducted a systematic feature engineering exploration across five directions: size transformations, species normalization, ordinal care encoding, care interaction indices, and combined redundant features. Across 14 experiments, no engineered feature exceeded the 0.01 F1/F2 improvement threshold. The best performance is `care_index` from ordinal care encoding, calculated as `steward_num` x `health_num`, representing a combined care quality score from 0 (neglected, unhealthy) to 9 (well-cared-for, healthy). It yielded $\Delta F1 = +0.0020$ and $\Delta F2 = +0.0017$ at the F1-optimal and F2-optimal thresholds, a gain too small to justify the added complexity. The worst-performing variant was the combined care indices, degrading performance by $\Delta F1 = -0.0073$ and $\Delta F2 = -0.0057$.

We also performed a leakage audit, removing 8 root/trunk/branch problem features except `root_stone`. Following Occam's razor, this simplified model performs equiv-

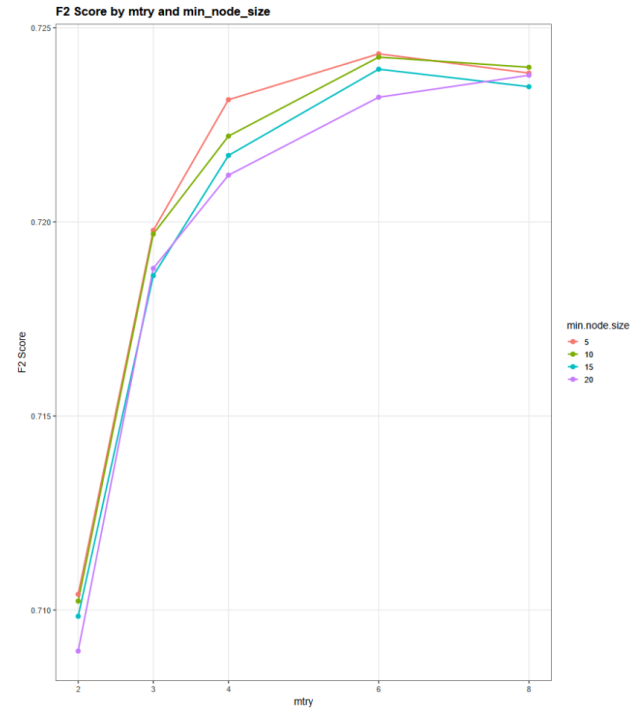


Figure 3. Results of Random Forest hyperparameter tuning. The model generally favored a higher mtry and a lower min.node.size.

mtry	min.node.size	F2-Score
6	5	0.7243
6	10	0.7242
8	10	0.7240
6	15	0.7239
8	5	0.7238

Table 3. Results of Random Forest hyperparameter tuning. Top 5 hyperparameters, sorted by F2-score.

alently to the original ($\Delta F1 = -0.0003$ and $\Delta F2 = +0.0024$) while eliminating any data leakage concern. Unfortunately, these results demonstrate that our Random Forest model might have possibly reached the signal ceiling of the current dataset. Further improvement requires external data sources rather than transformations of existing variables, a possible direction we are considering is soil data.

6.5. Random Forest Hyperparameter Tuning

All tuning runs used a fixed random seed of 123, 500 trees, F2-score optimized threshold, and applied class weights with the same 2.48 ratio used in the baseline. These are hyperparameter tuned after removing the root/trunk/branch problem features. To optimize the random forest model, we performed a grid search over two key hyperparameters:

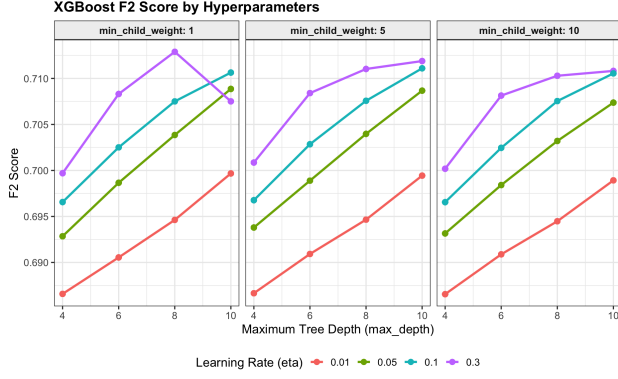


Figure 4. Results of XGBoost hyperparameter tuning. The model favored a lower min_child_weight, a higher max_depth, and a higher eta.

eta	min_child_weight	max_depth	F2-Score
0.3	1	8	0.7129
0.3	5	10	0.7119
0.1	5	10	0.7111
0.3	5	8	0.7110
0.3	10	10	0.7108

Table 4. Results of XGBoost hyperparameter tuning. Top 5 hyperparameters, sorted by F2-score.

the number of features sampled at each split (mtry) and the minimum number of observations required in a leaf node (min.node.size). We searched over mtry values of 2, 3, 4, 6, 8 and min.node.size values of 5, 10, 15, 20, evaluated on the development set using F2 score. The model performed best with a moderately high mtry of 6 suggesting that considering a larger subset of features at each split helps the model identify the right signals without overfitting to any single one. In contrast to the intermediate report, which optimized for F1 and favored larger node sizes, optimizing for F2 led to a best min.node.size of 5. Smaller leaf nodes allow the trees to make more specific predictions, which increases recall at the cost of some precision, a tradeoff that aligns with our goal of catching as many damaged trees as possible. The optimal configuration was found at mtry = 6 and min.node.size = 5, yielding an F2 score of 0.7243 on the dev set, as shown in Figure 3 and Table 3.

6.6. XGBoost Hyperparameter Tuning

All tuning runs used a fixed random seed of 123, 500 trees, F2-score optimized threshold, and applied class weights with the same 2.48 ratio used in the baseline. These are hyperparameter tuned after removing the root/trunk/branch problem features. To optimize the XGBoost model, we

performed a grid search over five hyperparameters: learning rate (eta), maximum tree depth (max_depth), fraction of training samples used per tree (subsample), fraction of features sampled per tree (colsample_bytree), and minimum number of samples required in a leaf (min_child_weight). We also varied the number of boosting rounds (nrounds). The grid covered eta values of 0.01, 0.05, 0.1, 0.3, max_depth values of 4, 6, 8, 10, subsample and colsample_bytree values of 0.6, 0.8, 1.0, min_child_weight values of 1, 5, 10, and nrounds values of 100, 200, 300, 500, resulting in 1,728 combinations.

Each configuration was evaluated on the development set using F2 score. We found that adjusting subsample and colsample_bytree hyperparameters yielded negligible change in F2. The most significant hyperparameters were min_child_weight, eta, and max_depth. The optimal configuration was found at max_depth = 8, min_child_weight = 1, and eta = 0.3 yielding an F2 score of 0.7129 on the dev set, as shown in Figure 4 and Table 4.

6.7. Clustering

We implemented feature transformation of the longitude/latitude data with the goal of reducing overfitting due to its high permutation importance in the random forest model. This was achieved using soft clustering from HDBSCAN with parameter selection by the DBOpt package (J. Hammer & Kaufman, 2025; L. McInnes & Astels, 2017). Rather than a data preprocessing, we treat this as a feature learning step to prevent training from using features affected by out-of-sample data. To better represent local patterns, we ran DBOpt separately on each borough with the exception of Brooklyn and Queens to avoid arbitrary separations by their land border.

Initial tests with DBOpt showed that roughly 65% of coordinates were labeled as noise for their hard cluster assignment, making the cluster label unsuitable as the sole feature due to this imbalance. Soft clustering remedies this by assigning a vector of probabilities to each observation where the i-th entry represents the probability of being a member of the i-th cluster. From these vectors we designed 17 features: (1) IDs of the top 5 clusters along with their probabilities (10 features total), (2) Hard cluster label by borough e.g. bq_hard_0012, si_noise, (3) Strength of the hard cluster assignment, (4) Difference between the highest and second-highest probabilities, called the margin, (5) Sum of soft cluster probabilities, called cluster mass, and one minus this sum, called off-cluster mass, (6) Normalized entropy of the soft cluster probabilities, measuring how concentrated or spread out the point’s cluster membership is across clusters.

Replacing the longitude/latitude pairs with these features, we ran another random forest grid search of the mtry and min.node.size hyperparameters which resulted in an AUC

Model	Accuracy	Precision	Recall	F2-Score
LR	0.4628	0.3387	0.9240	0.6867
RF	0.5814	0.3984	0.9121	0.7243
XGBoost	0.5524	0.3813	0.9101	0.7129

Table 5. Performance metrics of the hyperparameter-tuned models on the dev set. Random Forest (RF) outperformed both Logistic Regression (LR) and XGBoost.

of 0.561 and F2 score of 0.673, both of which were lower than before the transformation. There are a few possible explanations for this. While soft clustering was used to gain information for noise points, it's possible that many noise points still had a very low cluster mass, making them poorly explained by the learned clusters. Also, HDBSCAN optimizes solely for spatial density and therefore is unable to account for any other factors that might affect damage risk. Lastly, it's possible that the transformed features were simply unable to match the predictive weight of the raw features while generalizing more.

The clustering process was primarily limited by the HDBSCAN parameter selection. The DBOpt package, while representing a novel and highly accurate method for parameter selection, nonetheless had notable performance and compatibility issues. Significant time was spent switching it's HDBSCAN implementation to use McInnes' high performance version, as well as multiple smaller fixes bringing DBOpt up to date with its dependencies (McInnes & Healy, 2017). It's possible that further experiments with the transformed features might have revealed a better model, but unfortunately could not be completed.

6.8. Final Model Comparison

Across all model families, the hyperparameter-tuned random forest produced the strongest overall performance. On the development set, it achieved an F2-score of 0.7243, outperforming both logistic regression and XGBoost. While logistic regression assumes a linear log-odds relationship, the random forest ensemble more effectively captures complex non-linear interactions between the features.

Surprisingly, the tuned XGBoost model fell short of random forest despite the fact that it usually performs well on structured tabular data. Sidewalk damage is likely influenced by many unobserved external factors, such as soil conditions, sidewalk age, pavement material, and prior repairs. Because random forest averages across many independently trained trees, it may be more robust to this kind of noise than a boosting model that sequentially focuses on difficult-to-classify observations.

Accuracy	Precision	Recall	F2-Score
0.5829	0.4012	0.9077	0.7247

Table 6. Performance metrics of the hyperparameter-tuned random forest model on the test set.

		Predicted	
		Damage	No Damage
Actual	Damage	17,065 (TP)	1,735 (FN)
	No Damage	25,466 (FP)	20,951 (TN)

Table 7. Confusion Matrix of the final random forest model on the test set.

6.9. Final Model Metrics

Tables 6 and 7 detail the final random forest model's performance on the test set. By intentionally optimizing for the F2 score, the model successfully captures over 90% of actual sidewalk damage (17,065 true positives), missing only 1,735 cases. As shown in the confusion matrix, this high recall comes at the expense of precision, resulting in 25,466 false positives. In the context of NYC municipal operations, this trade-off is strictly by design. It ensures the vast majority of hazardous sidewalks are flagged for repair, accepting the administrative cost of false alarms over the safety and liability risks of unaddressed damage.

7. Conclusion

This project set out to predict sidewalk damage from NYC street tree data using only tree characteristics available before an inspection. Of the models we tested, random forest performed best, achieving an F2 score of 0.7247 and a recall of 90.77% on held-out test data.

Several approaches we expected to help, including feature engineering and XGBoost, did not meaningfully improve performance. Part of this comes down to the nature of the dataset itself. The 2015 Street Tree Census spans the entire city of New York, covering hundreds of thousands of trees across highly varied neighborhoods, soil conditions, and infrastructure. At that scale, sidewalk damage has a large random component that no set of tree-level features can fully explain. This may also be why random forest outperformed XGBoost. A model that handles noise well tends to do better than one that tries to find deeper structure when that structure does not exist.

A natural next step would be to incorporate external data such as pavement type, soil composition, or infrastructure age, which may capture some of the variation the current features cannot.

References

- Belyaev, D. Small project i worked on — 2015 nyc tree census, 2023.
- Francis, J. K., Parresol, B. R., and de Patino, J. M. Probability of damage to sidewalks and curbs by street trees in the tropics. *Journal of Arboriculture*, 22(4), 1996.
- J. Hammer, A. D. and Kaufman, L. Bayesian optimized parameter selection for density-based clustering applied to single molecule localization microscopy. *Communications Biology*, 8(902), 2025.
- L. McInnes, J. H. and Astels, S. hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software*, 2(11):205, 2017.
- Lin, J. and Wang, Q. Are street tree inequalities growing or diminishing over time? the inequity remediation potential of the milliontreesnyc initiative. *Journal of Environmental Management*, 285:112–207, 2021.
- McInnes, L. and Healy, J. Accelerated hierarchical density based clustering. In *Data Mining Workshops (ICDMW), 2017 IEEE International Conference on*, pp. 33–42. IEEE, 2017.
- NYC OpenData. 2015 street tree census - tree data, 2015. Accessed: 2026-03-04.
- Stevens, N. Tree troubles – predicting sidewalk damage resulting from trees in nyc, 2016.